Syllabus for Embedded Operating Systems

PGCP-ESD February 2026 Batch
Duration: 120 hours (42 theory hours + 48 lab hours + 30 hours self study)
Objective: To introduce students to the fundamental concepts of operating systems and facilitate hands-on experience with operating system functionalities.
Prerequisites: Working knowledge of Windows and Linux, familiarity with programming.
Evaluation: 100 Marks (Theory exam – 40%, Lab exam – 40%, Internals – 20%)

Textbook:

- Mastering Embedded Linux Programming, Second Edition, Chris Simmonds, Packt Publishing

  or

- Building Embedded Linux Systems, Karim Yaghmour, Jon Masters, Gilad Ben-Yossef, Philippe Gerum, OReilly Publications

References:

- https://bootlin.com/doc/training/embedded-linux/

Session 1: Introduction to Operating Systems (2T + 2L +1SL)

Lecture:

- Definition of an Operating System
- Basic Operating System Terminology
- Types of Operating Systems: Batch Processing, Time-Sharing, Distributed, Real-Time
- Overview of Windows and Linux Operating Systems

Lab:

- Introduction to Command Line Interface (CLI) in Windows and Linux
- Basic File Operations in Windows and Linux

Session 2: Operating System Structure (2T + 2L +1SL)

Lecture:

- Layers of an Operating System: Kernel, Shell, File System, Device Drivers
- User Mode vs Kernel Mode
- System Calls and APIs
- Process Management and Scheduling

Lab:

- Exploring Process Management Commands in Windows and Linux
- Understanding Process Scheduling Algorithms

Sessions 3 : Operating System Administration (2T + 2L +1SL)

Lecture:

- System Boot Process
- System Initialization and Shutdown
- Backup and Recovery Strategies
- System Monitoring and Maintenance

Lab:

- Exploring Boot Process and System Initialization in Windows and Linux
- Implementing Backup and Recovery Procedures

Sessions 4&5: Process Management (4T + 5L +4SL)

Lecture:

- Process Concept: Process States, PCB, Context Switching
- Deadlocks: Prevention, Avoidance, Detection, Recovery

Lab:

- Implementing Process Synchronization Techniques in C Programs
- Simulating Deadlock Scenarios and Analyzing Solutions

Session 6: Operating System Performance (2T + 2L +1SL)

Lecture:

- Performance Metrics: CPU Utilization, Throughput, Response Time, Latency
- Performance Tuning Techniques
- Benchmarking and Profiling Tools

Lab:

- Monitoring System Performance in Windows and Linux
- Analyzing Performance Metrics and Identifying Bottlenecks

Session 7: System Calls & Library Calls (2T + 2L +2SL)

Lecture :

- Overview of System calls and their usage in programming
- Differences between Library calls and System calls.

Lab :

- Implementation of Basic Linux System Calls using C
- Comparative Study of System Calls and Library Calls

Sessions 8&9: Process Synchronization (4T + 5L +3SL)

Lecture :

- Process Synchronization: Mutual Exclusion, Semaphores, Monitors

Lab :

- Implementation of Mutual Exclusion using Mutex Locks
- Implementation of Producer–Consumer Problem using Semaphores

Sessions 10&11: Inter Process Communication (4T + 5L +3SL)

Lecture :

- Interprocess communication between related and unrelated process

Lab :

- Inter-Process Communication between Related Processes using Pipes
- Inter-Process Communication between Unrelated Processes using FIFOs

Sessions 12&13: Memory Management (4T + 5L +4SL)

Lecture:

- Memory Hierarchy: Registers, Cache, Main Memory, Secondary Memory

- Memory Allocation Strategies: Contiguous Allocation, Paging, Segmentation
- Virtual Memory: Demand Paging, Page Replacement Algorithms, Thrashing

Lab:

- Understanding Memory Management Commands in Windows and Linux
- Simulating Paging and Page Replacement Algorithms

Sessions 14&15: Virtual Memory (4T + 5L +3SL)

Lecture:

- Importance of virtual memory
- Demand Paging, Trashing, Page fault.

Lab :

- Simulation of Demand Paging and Page Replacement Algorithms
- Analysis of Page Faults and Thrashing in Virtual Memory Systems

Session 16: File Systems (2T + 2L +1SL)

Lecture:

- File System Concepts: File, Directory, File Operations
- File System Implementation: File Allocation Methods, Directory Structure
- File System Security and Permissions

Lab:

- Exploring File System Commands in Windows and Linux
- Creating and Managing Files and Directories

Session 17: I/O Systems (2T + 2L +1SL)

Lecture:

- I/O Devices and Controllers
- I/O Operations: Blocking vs Non-Blocking, Synchronous vs Asynchronous
- Device Management: Device Drivers, Device Controllers, Device Queues

Lab:

- Understanding I/O Management Commands in Windows and Linux
- Simulating I/O Operations and Device Management

Sessions 18 & 19: Operating System Security (4T + 5L +3SL)

Lecture:

- Security Threats and Attacks
- Authentication and Authorization
- Access Control Lists (ACLs) and Permissions
- Encryption and Cryptography

Lab:

- Configuring User Accounts and Permissions in Windows and Linux
- Implementing Encryption and Decryption Techniques

Session 20 : Networking and Distributed Systems (2T + 2L +1SL)

Lecture:

- Network Fundamentals: OSI Model, TCP/IP Protocol Suite
- Client-Server Architecture
- Distributed Operating Systems: Distributed File Systems, Distributed Process Management
- Cloud Computing: Virtualization, Service Models (IaaS, PaaS, SaaS)

Lab:

- Exploring Networking Commands and Tools in Windows and Linux
- Setting up a Simple Client-Server Application

Session 21 : Socket Programming (2T + 2L +1SL)

Lecture :

- Usage of sockets
- Introduction to socket programming

Lab :

- Implementation of TCP Client–Server Communication using Sockets
- Implementation of UDP Client–Server Communication using Sockets

Self Learning Topics

1. Embedded Linux Internals

- Linux boot sequence (ROM → bootloader → kernel → init)
- Role of bootloaders (U-Boot basics – conceptual)

- Kernel vs user-space responsibility split
- Root filesystem structure

2. Process vs Thread (Deeper View)

- Threads vs processes comparison
- POSIX threads (conceptual understanding)
- Context switch cost comparison
- When to use processes vs threads

3. Scheduling – Practical Perspective

- Real-time vs normal Linux scheduling
- Priority inversion problem
- Scheduler tuning concepts
- CPU affinity basics

4. Memory Internals (Beyond Theory)

- Stack growth direction
- Heap allocation behavior
- Memory fragmentation issues
- Page cache vs buffer cache
- Memory leaks in long-running systems

5. Embedded Linux File Systems

- Flash file systems overview (conceptual):
  - JFFS2, YAFFS, UBIFS
- Read-only vs read-write rootfs
- Mount process basics

6. Signals & Process Control

- Linux signals overview
- Signal handling concepts
- Zombie and orphan processes
- Process lifecycle debugging

7. Inter-Process Communication – Practical Awareness

- Pipes vs FIFOs use cases
- Shared memory vs message queues trade-offs

- Synchronization pitfalls in IPC

8. Embedded Linux I/O Optimization

- Polling vs interrupt-driven I/O
- Blocking vs event-driven design
- Select / poll / epoll (conceptual)

9. Power & Performance in Embedded OS

- CPU frequency scaling concepts
- Power-aware process design
- Reducing wakeups
- Energy vs performance trade-offs

10. Fault Handling & Reliability

- Watchdog usage from OS perspective
- Handling application crashes
- Restart strategies (respawn concepts)
- Logging strategies in embedded systems

11. Debugging Embedded OS Issues

- Debugging without GUI tools
- Log-based debugging
- Identifying deadlocks and hangs
- Kernel vs user-space bug identification

12. Build Systems Awareness

- Cross-compilation basics
- Toolchain components (gcc, linker, libc)
- Why build systems matter in embedded Linux

13. Security Beyond Basics

- Secure boot concept
- Principle of least privilege
- Running services as non-root
- Attack surface reduction

14. Transition Topics

- Embedded OS → RTOS comparison
- Embedded OS → Device Drivers readiness
- Embedded OS → Embedded Linux applications

15. Industry & Interview Readiness

- Common Embedded OS interview questions
- Explaining OS concepts using real examples
- Debug-based OS interview problems
- Reading Linux man pages effectively